

CSS Display-Mode @media Rule

Complete Guide: From Basics to Advanced

CSS Guide

May 10, 2026

Contents

1	Introduction	2
1.1	What is Display-Mode @media?	2
1.2	Display-Mode vs. Regular Media Queries	2
2	Core Display Modes	3
2.1	Available Display Modes	3
2.2	Fullscreen Mode	3
2.2.1	When It's Used	3
2.2.2	Characteristics	3
2.2.3	Example: Game Interface	4
2.3	Standalone Mode	4
2.3.1	When It's Used	4
2.3.2	Characteristics	5
2.3.3	Example: PWA App Styling	5
2.4	Minimal-UI Mode	5
2.4.1	Overview	5
2.4.2	Example: Custom Navigation App	6
2.5	Browser Mode	6
2.5.1	Overview	6
3	Practical Implementation	7
3.1	Complete Multi-Mode CSS Example	7
3.2	Adaptive Navigation Pattern	9
3.2.1	Show Back Button Only in Standalone Mode	9
3.2.2	Responsive Sidebar	9
3.3	Safe Area Insets	10
3.3.1	Understanding Safe Areas	10
3.3.2	Implementation Example	10
4	PWA Configuration	11
4.1	Setting Display Mode in manifest.json	11
4.1.1	Complete Manifest Configuration	11
4.1.2	Key Manifest Properties	12
5	Real-World Use Cases	13
5.1	Use Case 1: Mobile Productivity App	13
5.1.1	Requirements	13
5.1.2	Implementation	13

5.2	Use Case 2: Fullscreen Gaming App	14
5.2.1	Requirements	14
5.2.2	Implementation	14
5.3	Use Case 3: Video Streaming App	15
5.3.1	Requirements	15
5.3.2	Implementation	15
6	Browser Support	16
6.1	Current Support Status	16
6.2	Fallback Strategy	16
7	Best Practices	17
7.1	Key Recommendations	17
7.1.1	1. Always Provide Fallback Styles	17
7.1.2	2. Account for Safe Areas	17
7.1.3	3. Combine with Other Media Queries	17
7.1.4	4. Test All Display Modes	18
7.1.5	5. Optimize Touch Targets	18
8	Common Mistakes & Solutions	19
8.1	Mistake 1: Hardcoding Padding	19
8.2	Mistake 2: Ignoring Fallback Styles	19
8.3	Mistake 3: Hiding Essential Content	19
9	Summary & Key Takeaways	21
9.1	When to Use Display-Mode Queries	21
9.2	When NOT to Use Display-Mode Queries	21
9.3	Quick Reference: Display Modes	21
9.4	Properties Recap	22

Chapter 1

Introduction

1.1 What is Display-Mode @media?

The `display-mode` media feature in CSS allows you to target different display modes of your web application. This is especially useful for Progressive Web Apps (PWAs) that can run in different modes: standalone, fullscreen, or in a browser window.

When you build a PWA, users can install your app and run it in different ways. The `display-mode` media query helps you adapt your CSS based on how the app is currently running.

```
/* Basic syntax */
@media (display-mode: standalone) {
  /* Styles for standalone mode */
}
```

1.2 Display-Mode vs. Regular Media Queries

Unlike traditional media queries that check screen size or device capabilities, the `display-mode` feature checks **how** the application is being displayed. This fundamental difference makes it perfect for PWA optimization.

- **Screen Media Queries:** Check viewport size, orientation, resolution
- **Display-Mode Media Queries:** Check how the app is running (browser, standalone, fullscreen)
- **Combined Approach:** Use both together for comprehensive responsive design

Chapter 2

Core Display Modes

2.1 Available Display Modes

There are four main display modes you can target with the display-mode media feature:

Mode	Description	Browser UI
fullscreen	App takes up entire screen with no browser UI	Hidden
standalone	App looks and feels like a native app	Minimal (status bar only)
minimal-ui	Compact browser UI with navigation buttons	Minimal
browser	Traditional browser experience (fallback)	Full

2.2 Fullscreen Mode

2.2.1 When It's Used

Fullscreen mode is ideal for:

- Gaming applications and interactive experiences
- Video players and media consumption apps
- Presentations and immersive content
- Augmented Reality (AR) experiences
- Kiosk applications

2.2.2 Characteristics

- No browser UI visible at all
- App occupies entire screen ($100\text{vw} \times 100\text{vh}$)
- Status bar may still show on mobile devices
- Perfect for full-screen experiences
- Maximum screen real estate for content

2.2.3 Example: Game Interface

```
@media (display-mode: fullscreen) {  
  body {  
    margin: 0;  
    padding: 0;  
    width: 100vw;  
    height: 100vh;  
    overflow: hidden;  
  }  
  
  .ui-navbar {  
    display: none;  
  }  
  
  .ui-sidebar {  
    display: none;  
  }  
  
  .game-container {  
    width: 100%;  
    height: 100%;  
    background: linear-gradient(135deg, #667eea, #764ba2);  
  }  
  
  .game-controls {  
    position: absolute;  
    bottom: 20px;  
    right: 20px;  
    z-index: 100;  
  }  
}
```

Result: When the app runs in fullscreen mode, navigation elements disappear and the game controls appear in the corner for optimal viewing.

2.3 Standalone Mode

2.3.1 When It's Used

Standalone mode is the most common for PWAs:

- Mobile app installations on home screen
- Installed desktop applications
- Native-looking web applications
- Productivity and utility apps
- Most PWAs default to this mode

2.3.2 Characteristics

- Looks and feels like a native application
- No address bar or traditional browser controls
- Can show status bar on mobile devices
- Status bar displays time, battery, signal strength
- Most familiar to mobile app users

2.3.3 Example: PWA App Styling

```
@media (display-mode: standalone) {  
  body {  
    margin: 0;  
    padding: 0;  
    padding-top: env(safe-area-inset-top);  
    background: #ffffff;  
    font-family: -apple-system, BlinkMacSystemFont,  
                 'Segoe UI', Roboto, sans-serif;  
  }  
  
  .header {  
    padding: 20px;  
    background: linear-gradient(135deg, #667eea, #764ba2);  
    color: white;  
    padding-top: calc(20px + env(safe-area-inset-top));  
  }  
  
  .footer {  
    position: fixed;  
    bottom: 0;  
    width: 100%;  
    background: rgba(0, 0, 0, 0.05);  
    border-top: 1px solid #e0e0e0;  
    padding-bottom: env(safe-area-inset-bottom);  
  }  
  
  .content {  
    padding-bottom: calc(80px + env(safe-area-inset-bottom));  
  }  
}
```

Result: When installed as a standalone app, the UI automatically accounts for safe areas (notches, rounded corners) and provides a native app experience.

2.4 Minimal-UI Mode

2.4.1 Overview

Minimal-UI mode provides a middle ground between fullscreen and standalone:

- Shows minimal browser controls (back, forward, reload buttons)
- No address bar visible
- Good for apps with their own navigation
- Primarily supported on Android browsers
- Limited support on other platforms

2.4.2 Example: Custom Navigation App

```
@media (display-mode: minimal-ui) {  
  .app-container {  
    padding-top: 10px;  
  }  
  
  .back-button {  
    display: none;  
  }  
  
  .navigation {  
    background: #f5f5f5;  
    border-bottom: 1px solid #e0e0e0;  
  }  
}
```

2.5 Browser Mode

2.5.1 Overview

Browser mode is the default fallback for traditional web browsing:

- Used when app is opened in regular browser
- All standard browser controls available
- Full address bar visible
- Traditional web experience
- Fallback for unsupported modes

Chapter 3

Practical Implementation

3.1 Complete Multi-Mode CSS Example

```
/* Default styles for browser mode */
body {
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana;
    padding: 20px;
    background: #ffffff;
    color: #333333;
}

.header {
    background: #667eea;
    color: white;
    padding: 15px;
    border-radius: 8px;
    margin-bottom: 20px;
}

.browser-notice {
    background: #e8f4f8;
    border-left: 4px solid #3498db;
    padding: 15px;
    margin-bottom: 20px;
}

/* Standalone Mode - Installed App */
@media (display-mode: standalone) {
    body {
        padding: 0;
        margin: 0;
        padding-top: env(safe-area-inset-top);
    }

    .header {
        padding: 25px;
        border-radius: 0;
        padding-top: calc(25px + env(safe-area-inset-top));
    }
}
```

```
.browser-notice {
    display: none;
}

main {
    padding: 20px;
}

}

/* Fullscreen Mode - Games/Media */
@media (display-mode: fullscreen) {
    body {
        margin: 0;
        padding: 0;
        width: 100vw;
        height: 100vh;
        overflow: hidden;
    }

    .header,
    .footer,
    .browser-notice {
        display: none;
    }

    main {
        width: 100%;
        height: 100%;
        padding: 0;
    }
}

/* Minimal-UI Mode */
@media (display-mode: minimal-ui) {
    body {
        margin: 0;
        padding-top: env(safe-area-inset-top);
    }

    .browser-notice {
        display: none;
    }
}

/* Responsive + Display Mode Combined */
@media (display-mode: standalone) and (max-width: 768px) {
    .sidebar {
        display: none;
    }

    main {
        width: 100%;
    }
}
```

```
}
```

3.2 Adaptive Navigation Pattern

3.2.1 Show Back Button Only in Standalone Mode

```
.back-button {  
    display: none;  
}  
  
@media (display-mode: standalone) {  
    .back-button {  
        display: inline-block;  
        background: #667eea;  
        color: white;  
        border: none;  
        padding: 10px 15px;  
        border-radius: 5px;  
        cursor: pointer;  
        font-weight: 600;  
    }  
  
    .back-button:hover {  
        background: #764ba2;  
    }  
}
```

3.2.2 Responsive Sidebar

```
.sidebar {  
    width: 250px;  
    background: #f0f0f0;  
    position: fixed;  
    left: 0;  
    top: 0;  
    height: 100vh;  
}  
  
/* Hide sidebar in fullscreen mode */  
@media (display-mode: fullscreen) {  
    .sidebar {  
        display: none;  
    }  
  
    main {  
        margin-left: 0;  
    }  
}  
  
/* Optimize sidebar for standalone */
```

```
@media (display-mode: standalone) {  
  .sidebar {  
    width: 100%;  
    position: static;  
    height: auto;  
    display: flex;  
    padding: 0 15px;  
    padding-top: env(safe-area-inset-top);  
  }  
}
```

3.3 Safe Area Insets

3.3.1 Understanding Safe Areas

Safe area insets are CSS environment variables that tell you about device features like notches and rounded corners:

Inset	Purpose
safe-area-inset-top	Space for notch or rounded corner at top
safe-area-inset-bottom	Space for home indicator or rounded corner
safe-area-inset-left	Space for rounded corner on left side
safe-area-inset-right	Space for rounded corner on right side

3.3.2 Implementation Example

```
@media (display-mode: standalone) {  
  body {  
    /* Add space for top notch */  
    padding-top: env(safe-area-inset-top);  
  
    /* Add space for bottom indicator */  
    padding-bottom: env(safe-area-inset-bottom);  
  
    /* Add space for left/right corners */  
    padding-left: env(safe-area-inset-left);  
    padding-right: env(safe-area-inset-right);  
  }  
  
  .header {  
    padding-top: calc(20px + env(safe-area-inset-top));  
  }  
  
  .footer {  
    padding-bottom: calc(20px + env(safe-area-inset-bottom));  
  }  
}
```

Chapter 4

PWA Configuration

4.1 Setting Display Mode in manifest.json

The display mode for your PWA is defined in the `manifest.json` file. This file tells the browser how your app should display when installed.

4.1.1 Complete Manifest Configuration

```
{
  "name": "My Awesome Application",
  "short_name": "MyApp",
  "description": "An amazing Progressive Web App
                  with offline capabilities",
  "start_url": "/",
  "display": "standalone",
  "scope": "/",
  "background_color": "#ffffff",
  "theme_color": "#667eea",
  "orientation": "portrait-primary",
  "icons": [
    {
      "src": "/icon-192x192.png",
      "sizes": "192x192",
      "type": "image/png",
      "purpose": "any"
    },
    {
      "src": "/icon-512x512.png",
      "sizes": "512x512",
      "type": "image/png",
      "purpose": "any"
    },
    {
      "src": "/maskable-192x192.png",
      "sizes": "192x192",
      "type": "image/png",
      "purpose": "maskable"
    }
  ],
}
```

```
"categories": ["productivity"],
"screenshots": [
  {
    "src": "/screenshot1.png",
    "sizes": "540x720",
    "type": "image/png",
    "form_factor": "narrow"
  }
]
}
```

4.1.2 Key Manifest Properties

Property	Type	Description
display	string	How to display the app (standalone, fullscreen, etc.)
name	string	Full application name
short_name	string	Short name for home screen
start_url	string	URL to load when app starts
theme_color	color	Browser UI color
background_color	color	Splash screen background

Chapter 5

Real-World Use Cases

5.1 Use Case 1: Mobile Productivity App

5.1.1 Requirements

- Installed on home screen
- Native app feel
- Account for safe areas
- Optimized touch interface

5.1.2 Implementation

```
@media (display-mode: standalone) {  
  body {  
    margin: 0;  
    padding-top: env(safe-area-inset-top);  
    font-size: 16px; /* Prevents mobile zoom on input */  
  }  
  
  header {  
    padding: 15px;  
    padding-top: calc(15px + env(safe-area-inset-top));  
    background: linear-gradient(135deg, #667eea, #764ba2);  
  }  
  
  button {  
    padding: 12px 20px;  
    font-size: 16px;  
    border-radius: 8px;  
    min-height: 44px; /* Touch target size */  
    min-width: 44px;  
  }  
  
  input {  
    padding: 12px;  
    font-size: 16px;  
  }  
}
```

```
        min-height: 44px;
    }
}
```

5.2 Use Case 2: Fullscreen Gaming App

5.2.1 Requirements

- Utilize entire screen
- Hide all UI elements
- Optimize performance
- Handle orientation changes

5.2.2 Implementation

```
@media (display-mode: fullscreen) {
    body {
        margin: 0;
        padding: 0;
        width: 100vw;
        height: 100vh;
        overflow: hidden;
        background: #000;
    }

    .navbar,
    .sidebar,
    .footer {
        display: none !important;
    }

    .game-canvas {
        width: 100%;
        height: 100%;
        display: block;
    }

    .game-hud {
        position: absolute;
        top: 10px;
        right: 10px;
        background: rgba(0, 0, 0, 0.7);
        color: white;
        padding: 10px;
        border-radius: 5px;
    }
}
```


5.3 Use Case 3: Video Streaming App

5.3.1 Requirements

- Support multiple display modes
- Fullscreen video playback
- Adaptive controls
- Minimal UI in fullscreen

5.3.2 Implementation

```
@media (display-mode: fullscreen) {  
  .player-controls {  
    opacity: 0;  
    transition: opacity 0.3s;  
  }  
  
  .player-container:hover .player-controls {  
    opacity: 1;  
  }  
  
  video {  
    width: 100%;  
    height: 100%;  
  }  
}  
  
@media (display-mode: standalone) {  
  .recommended-videos {  
    display: none;  
  }  
  
  .video-player {  
    max-width: 100%;  
  }  
}
```

Chapter 6

Browser Support

6.1 Current Support Status

Browser	Version	Status
Chrome/Edge (Desktop)	v73+	Full Support
Chrome (Android)	v54+	Full Support
Firefox (Desktop)	v87+	Full Support
Firefox (Android)	v68+	Full Support
Safari (macOS)	15.1+	Full Support
Safari (iOS)	15.1+	Limited Support
Samsung Internet	v5.0+	Full Support
Opera	v60+	Full Support

6.2 Fallback Strategy

For browsers without display-mode support, provide sensible defaults:

```
/* Default styles work everywhere */
body {
  margin: 0;
  padding: 20px;
  font-family: sans-serif;
}

/* Enhanced styles for supported browsers */
@media (display-mode: standalone) {
  body {
    padding: 0;
    /* Additional styling */
  }
}

/* No media query = graceful degradation */
```

Chapter 7

Best Practices

7.1 Key Recommendations

7.1.1 1. Always Provide Fallback Styles

Write your default CSS first, then enhance with media queries:

```
/* Works everywhere */
body { padding: 20px; }

/* Enhanced for standalone */
@media (display-mode: standalone) {
  body { padding: 0; }
}
```

7.1.2 2. Account for Safe Areas

Always use environment variables for notch/corner spacing:

```
@media (display-mode: standalone) {
  body {
    padding-top: env(safe-area-inset-top);
    padding-bottom: env(safe-area-inset-bottom);
  }
}
```

7.1.3 3. Combine with Other Media Queries

Mix display-mode with screen size queries:

```
@media (display-mode: standalone) and (max-width: 600px) {
  /* Styles for standalone mode on mobile */
}

@media (display-mode: fullscreen) and (orientation: landscape) {
  /* Styles for fullscreen landscape */
}
```

7.1.4 4. Test All Display Modes

Test your app in different contexts:

- **Browser:** Open website normally
- **Standalone:** Install PWA and open from home screen
- **Fullscreen:** Use fullscreen APIs or manifest display mode
- **Different devices:** Test on phones with/without notches

7.1.5 5. Optimize Touch Targets

In standalone mode, ensure touch targets are at least 44px × 44px:

```
@media (display-mode: standalone) {  
  button, a {  
    min-height: 44px;  
    min-width: 44px;  
    padding: 12px 16px;  
  }  
}
```

Chapter 8

Common Mistakes & Solutions

8.1 Mistake 1: Hardcoding Padding

Wrong:

```
@media (display-mode: standalone) {  
  body { padding-top: 50px; } /* Too much on normal phones */  
}
```

Correct:

```
@media (display-mode: standalone) {  
  body { padding-top: env(safe-area-inset-top); }  
}
```

8.2 Mistake 2: Ignoring Fallback Styles

Always ensure default styles work:

```
/* This breaks in non-supporting browsers */  
@media (display-mode: standalone) {  
  body { padding: 0; }  
}  
  
/* Better: provide default */  
body { padding: 20px; }  
@media (display-mode: standalone) {  
  body { padding: 0; }  
}
```

8.3 Mistake 3: Hiding Essential Content

Bad:

```
@media (display-mode: fullscreen) {  
  /* Hides critical navigation */  
  header { display: none; }  
}
```

Good:

```
@media (display-mode: fullscreen) {  
  /* Only hides decorative elements */  
  .decorative-banner { display: none; }  
  
  /* Keeps essential navigation accessible */  
  header { transform: translateY(-100%); }  
  header:hover { transform: translateY(0); }  
}
```

Chapter 9

Summary & Key Takeaways

9.1 When to Use Display-Mode Queries

Use display-mode media queries when:

- Building Progressive Web Apps (PWAs)
- Supporting multiple installation scenarios
- Optimizing UI for specific app contexts
- Improving native app-like feel
- Managing screen real estate efficiently

9.2 When NOT to Use Display-Mode Queries

Don't use display-mode when:

- You only target traditional web browsers
- The styling difference is negligible
- You need to detect specific screen sizes (use screen queries instead)
- The feature is not supported in target browsers

9.3 Quick Reference: Display Modes

Mode	Best For	Browser UI	Common Usage
fullscreen	Games, videos	None	5% of PWAs
standalone	General apps	Minimal	90% of PWAs
minimal-ui	Custom nav	Buttons only	3% of PWAs
browser	Web pages	Full	Traditional web

9.4 Properties Recap

CSS Syntax:

```
@media (display-mode: DISPLAY_MODE) {  
    /* Your styles here */  
}
```

Valid Values:

- fullscreen
- standalone
- minimal-ui
- browser

Combined Usage:

```
/* Multiple conditions */  
@media (display-mode: standalone) and (max-width: 768px) {  
    /* Your styles */  
}  
  
/* Negation */  
@media not (display-mode: browser) {  
    /* Your styles */  
}
```

Thank you for reading!

For more information, visit [MDN Web Docs](#) and [web.dev](#)